\newpage

Executive Summary

The Charissatics Ordering Application is a comprehensive digital marketplace platform designed specifically for the building materials industry in Nigeria. The system facilitates B2B and B2C transactions, enabling customers to order construction materials (granite, sand, gravel, etc.) directly from quarries with integrated logistics and payment solutions.

Key Features

- **Multi-user Support**: B2B (companies) and B2C (individual customers)
- **Product Management**: Comprehensive catalog of building materials
- **Dynamic Pricing**: Location-based pricing matrix with real-time calculations
- **Order Management**: End-to-end order processing with weight verification
- **Logistics Integration**: Vehicle hire and loading point management
- **Digital Wallet**: Prepaid wallet system for transactions
- **Admin Dashboard**: Complete administrative control panel
- **Mobile Ready**: RESTful API designed for mobile app integration

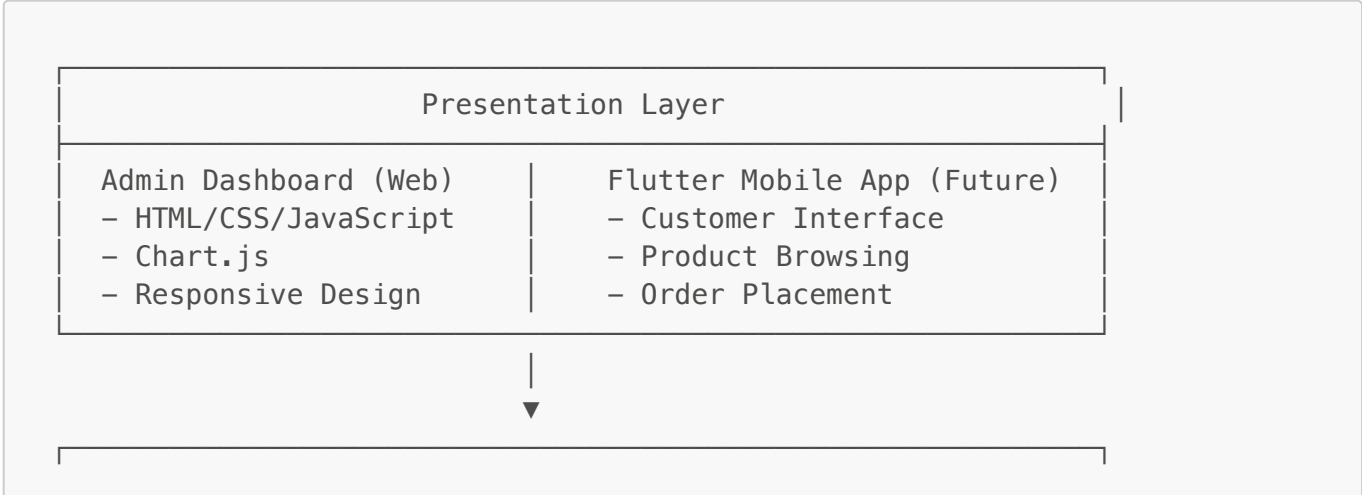
Technology Stack

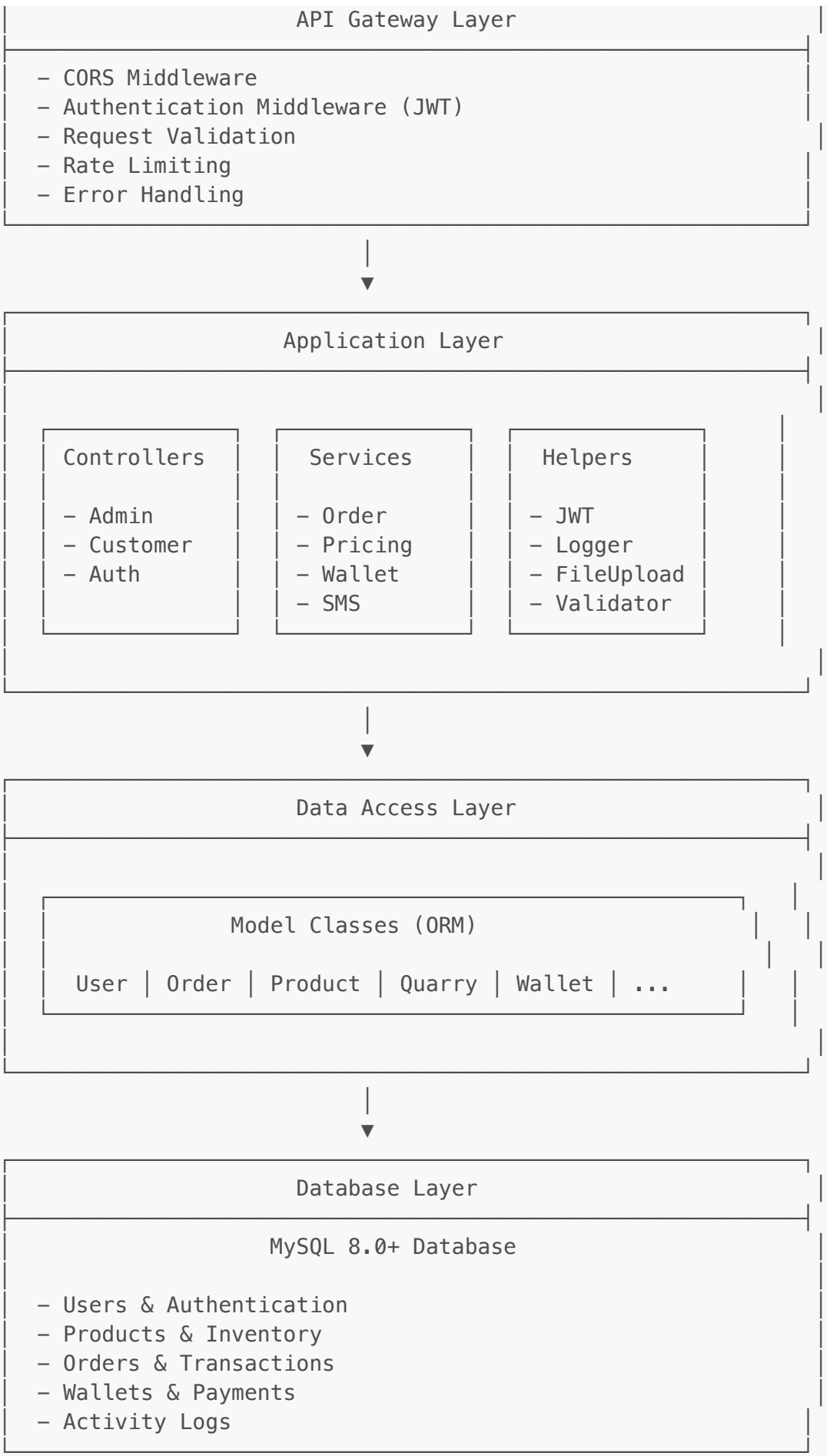
- **Backend**: PHP 8.0+ (Custom MVC Framework)
- **Database**: MySQL 8.0+
- **Authentication**: JWT (JSON Web Tokens)
- **Frontend (Admin)**: Vanilla HTML5, CSS3, JavaScript (ES6+)
- **API Architecture**: RESTful API
- **Mobile**: Ready for Flutter integration

\newpage

System Architecture

Architecture Overview





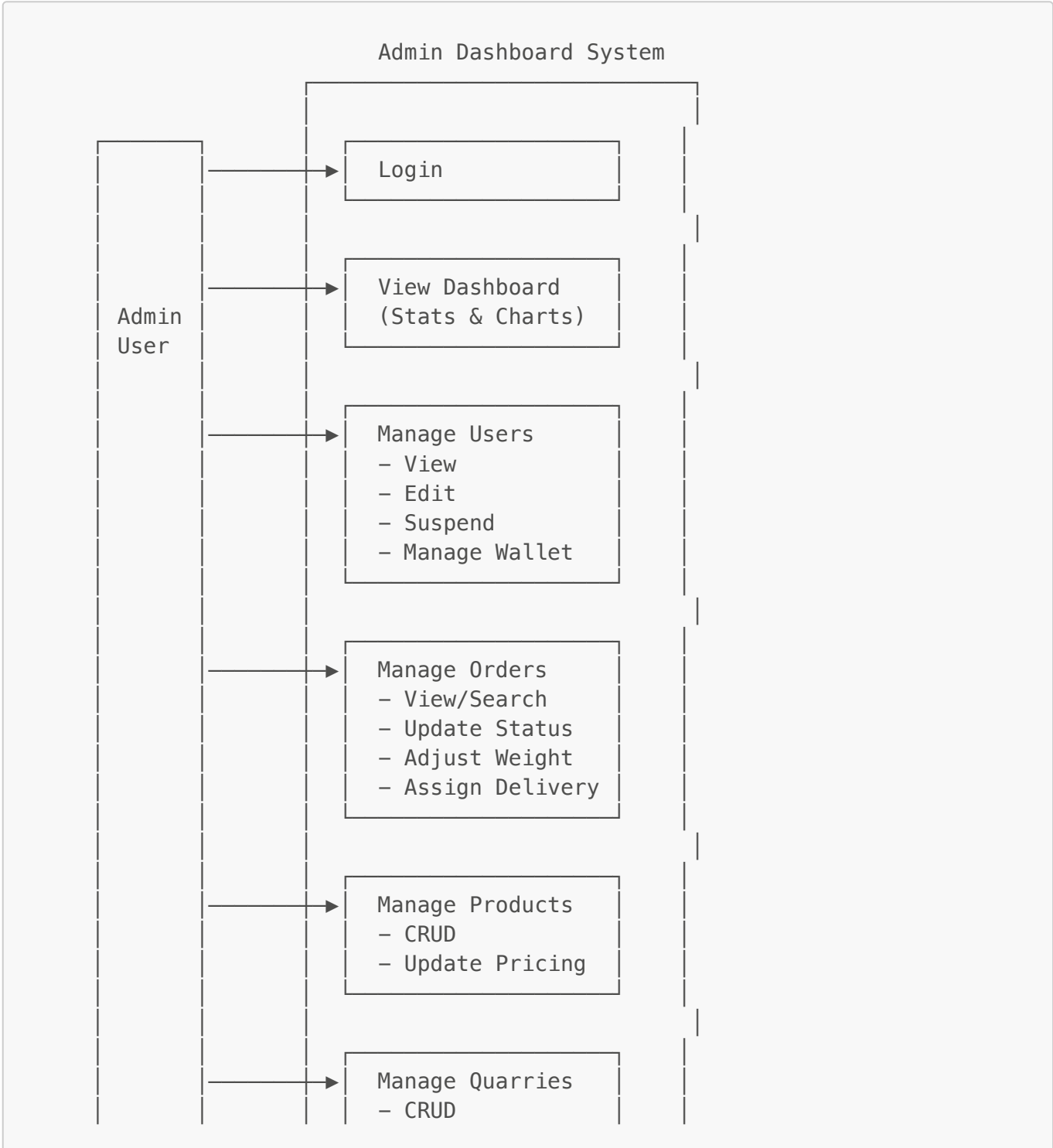
Design Patterns Used

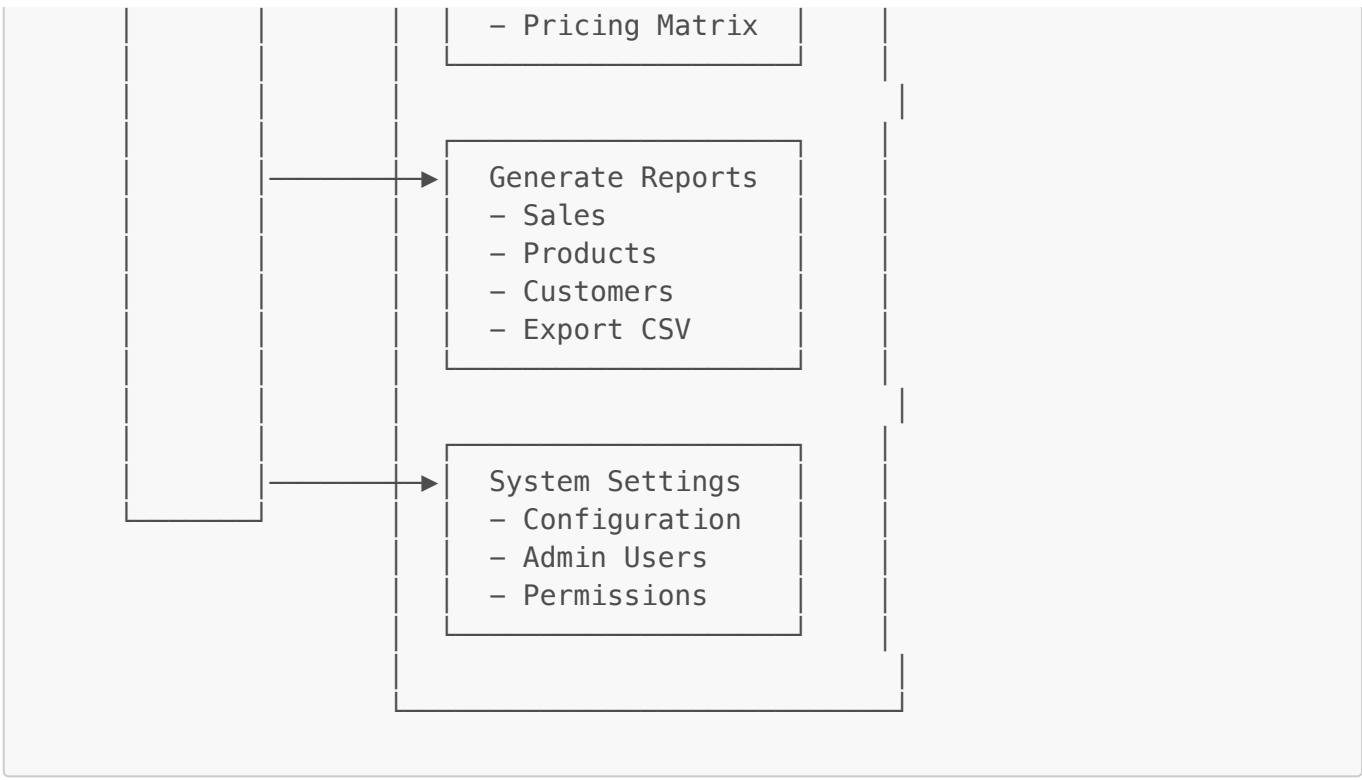
1. **MVC (Model-View-Controller)**: Core architectural pattern
2. **Singleton**: Database connection, Router
3. **Repository Pattern**: Model classes abstract database operations
4. **Middleware Pattern**: Request processing pipeline
5. **Factory Pattern**: Model instantiation
6. **Service Layer Pattern**: Business logic separation

\newpage

Use Case Diagrams

3.1 Admin Use Cases





Admin Use Case Descriptions

UC-A1: Admin Login

- **Actor**: Admin User
- **Precondition**: Admin account exists and is active
- **Main Flow**:
    1. Admin enters email and password
    2. System validates credentials
    3. System generates JWT tokens
    4. Admin is redirected to dashboard
- **Postcondition**: Admin is authenticated

UC-A2: Manage Users

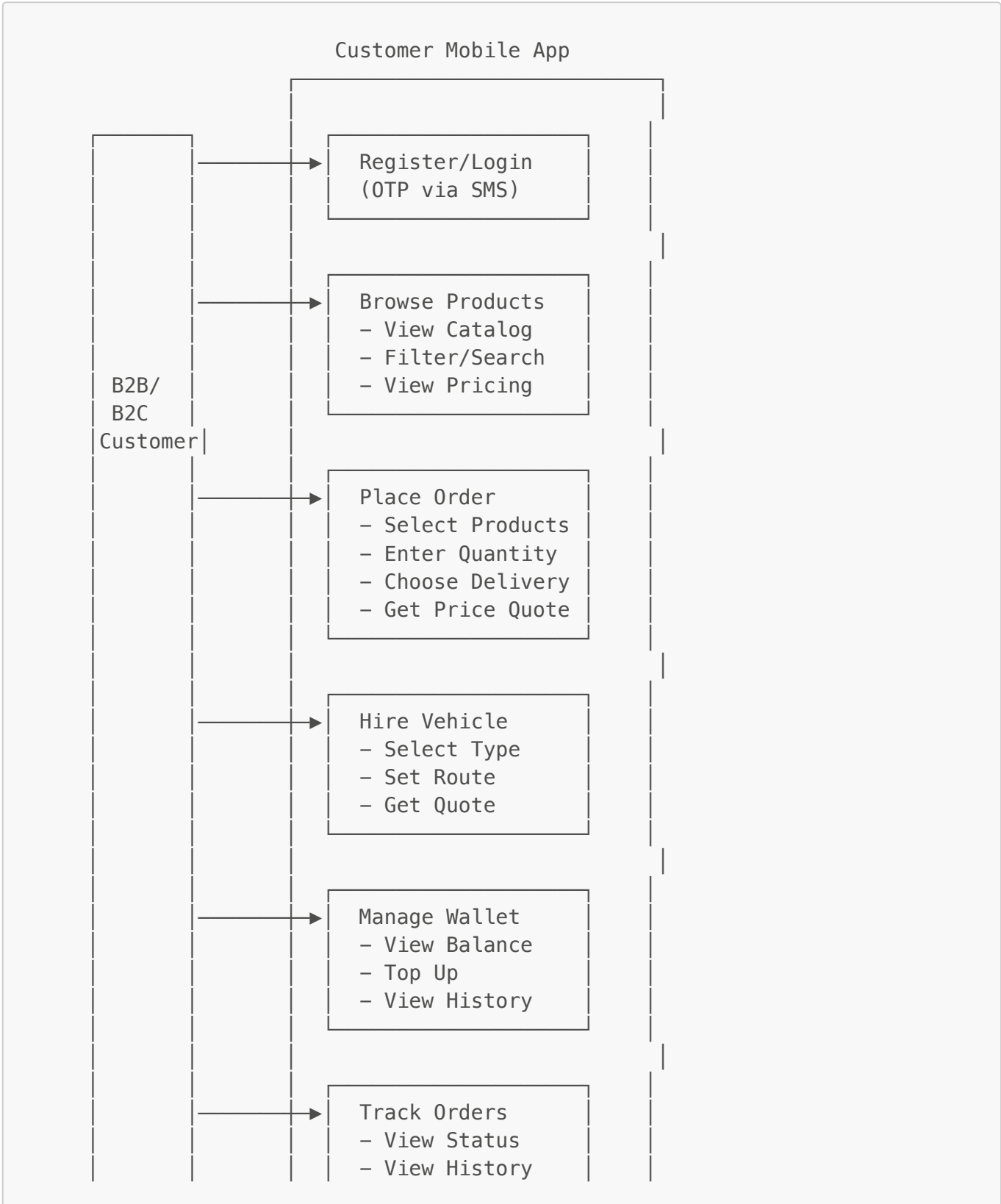
- **Actor**: Admin User
- **Precondition**: Admin is authenticated
- **Main Flow**:
    1. Admin views list of customers
    2. Admin can filter by type (B2B/B2C), status
    3. Admin can view user details
    4. Admin can suspend/activate accounts
    5. Admin can credit/debit wallet
- **Postcondition**: User management action is logged

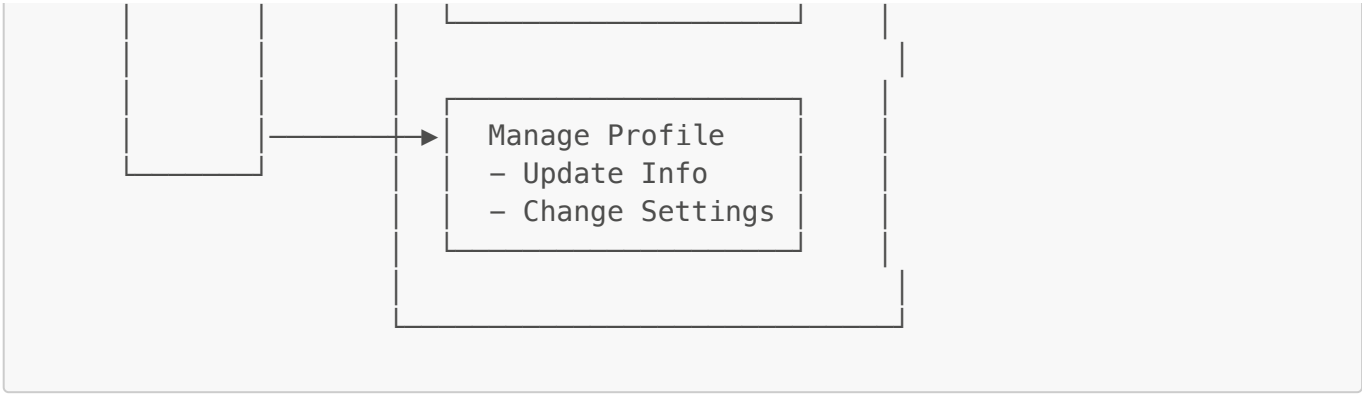
UC-A3: Manage Orders

- **Actor**: Admin User
- **Precondition**: Admin is authenticated

- **Main Flow**:
    1. Admin views orders list
    2. Admin can filter by status, date, customer
    3. Admin updates order status
    4. Admin adjusts actual weight after delivery
    5. System recalculates price if weight variance > threshold
- **Postcondition**: Order status updated, notifications sent

3.2 Customer Use Cases





Customer Use Case Descriptions

UC-C1: Register/Login with OTP

- **Actor**: Customer (B2B or B2C)
- **Precondition**: Customer has valid phone number
- **Main Flow**:
    1. Customer enters phone number
    2. System sends OTP via SMS
    3. Customer enters OTP
    4. System validates OTP
    5. System generates JWT tokens
- **Postcondition**: Customer is authenticated

UC-C2: Place Material Order

- **Actor**: Customer
- **Precondition**: Customer is authenticated
- **Main Flow**:
    1. Customer browses products
    2. Customer selects product and quantity
    3. Customer selects delivery location
    4. Customer chooses quarry (optional)
    5. System calculates price based on:
        - Product base price
        - Quantity (tonnage)
        - Distance to delivery location
        - Quarry pricing
    6. Customer confirms order
    7. System deducts from wallet
    8. Order is created
- **Postcondition**: Order created, quarry notified

UC-C3: Hire Vehicle

- **Actor**: Customer
- **Precondition**: Customer is authenticated
- **Main Flow**:

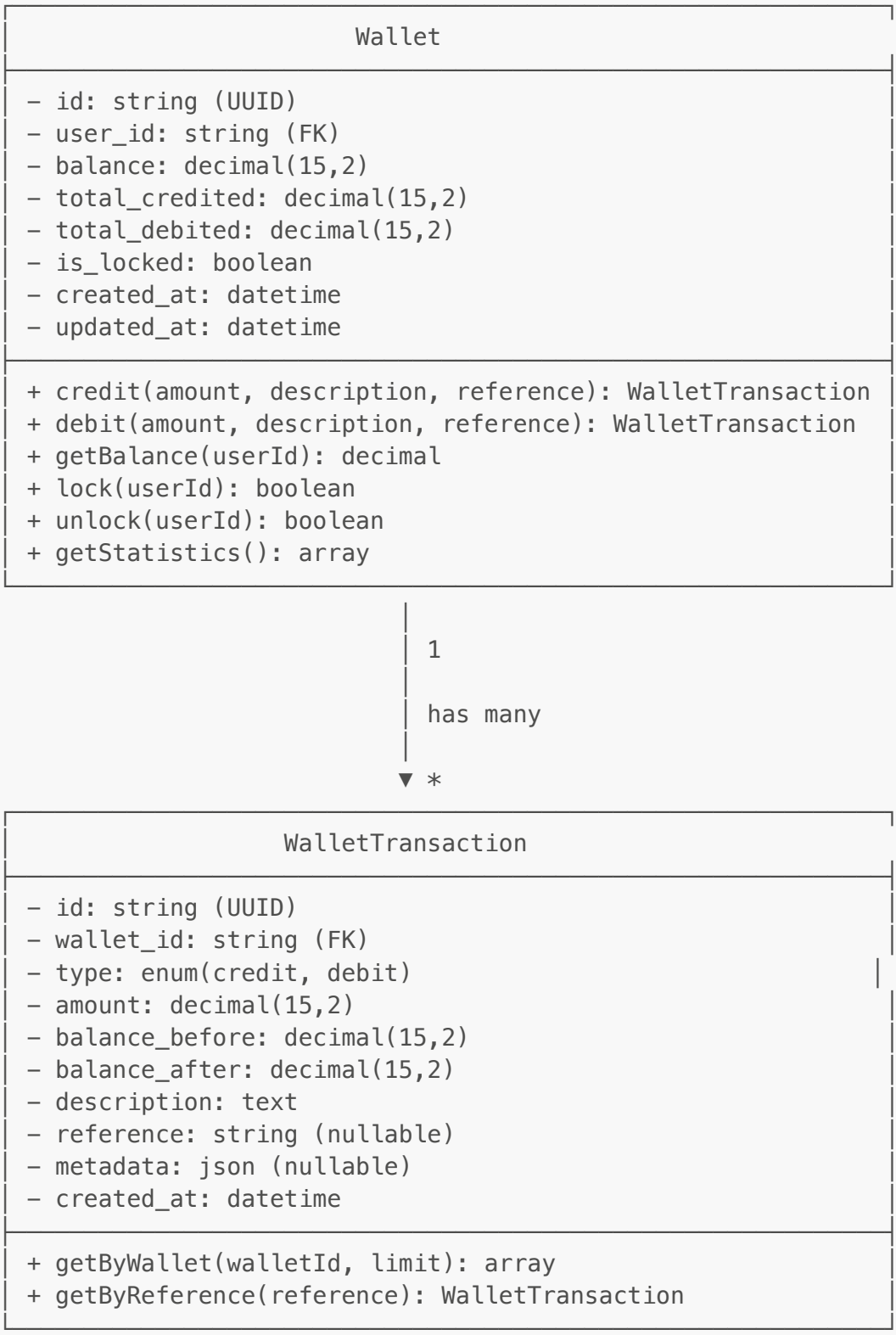
1. Customer selects vehicle type
2. Customer enters pickup location
3. Customer enters delivery location
4. System calculates distance
5. System calculates price based on:
    - Vehicle type rate per km
    - Distance
6. Customer confirms booking
7. System deducts from wallet

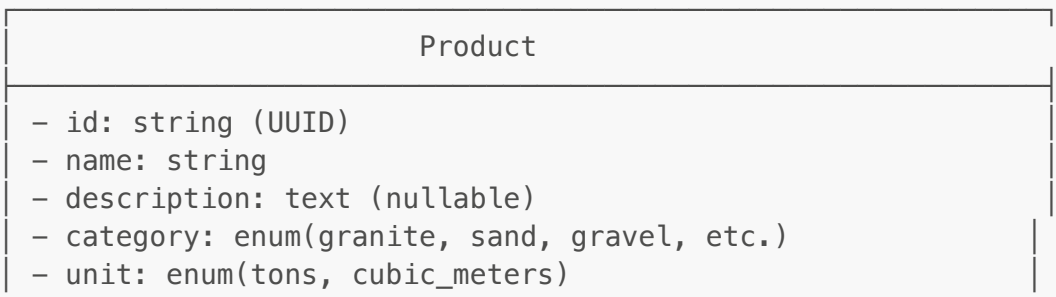
- **Postcondition**: Vehicle hire order created

\newpage

Class Diagram

4.1 Core Domain Classes


Wallet
- id: string (UUID)
- user_id: string (FK)
- balance: decimal(15,2)
- total_credited: decimal(15,2)
- total_debited: decimal(15,2)
- is_locked: boolean
- created_at: datetime
- updated_at: datetime
+ credit(amount, description, reference): WalletTransaction
+ debit(amount, description, reference): WalletTransaction
+ getBalance(userId): decimal
+ lock(userId): boolean
+ unlock(userId): boolean
+ getStatistics(): array
1
has many
*
WalletTransaction
- id: string (UUID)
- wallet_id: string (FK)
- type: enum(credit, debit)
- amount: decimal(15,2)
- balance_before: decimal(15,2)
- balance_after: decimal(15,2)
- description: text
- reference: string (nullable)
- metadata: json (nullable)
- created_at: datetime
+ getByWallet(walletId, limit): array
+ getByReference(reference): WalletTransaction


Product
- id: string (UUID)
- name: string
- description: text (nullable)
- category: enum(granite, sand, gravel, etc.)
- unit: enum(tons, cubic_meters)

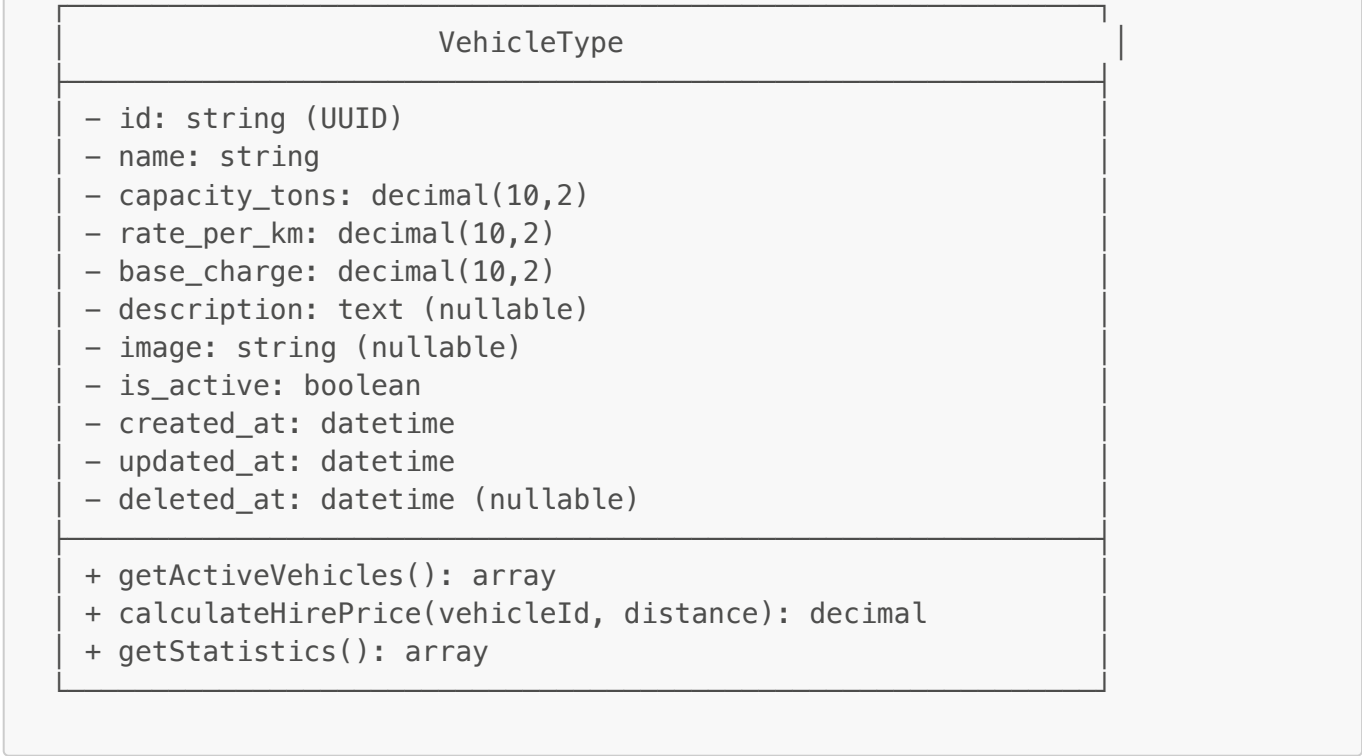
```
| - base_price_per_ton: decimal(10,2)                                  |
| - images: json (nullable)                                            |
| - specifications: json (nullable)                                    |
| - is_active: boolean                                                 |
| - created_at: datetime                                               |
| - updated_at: datetime                                               |
| - deleted_at: datetime (nullable)                                    |
|----------------------------------------------------------------------|
| + getActiveProducts(): array                                         |
| + getByCategory(category): array                                     |
| + updatePricing(id, price): boolean                                  |
| + getStatistics(): array                                             |
└──────────────────────────────────────────────────────────────────────┘
```

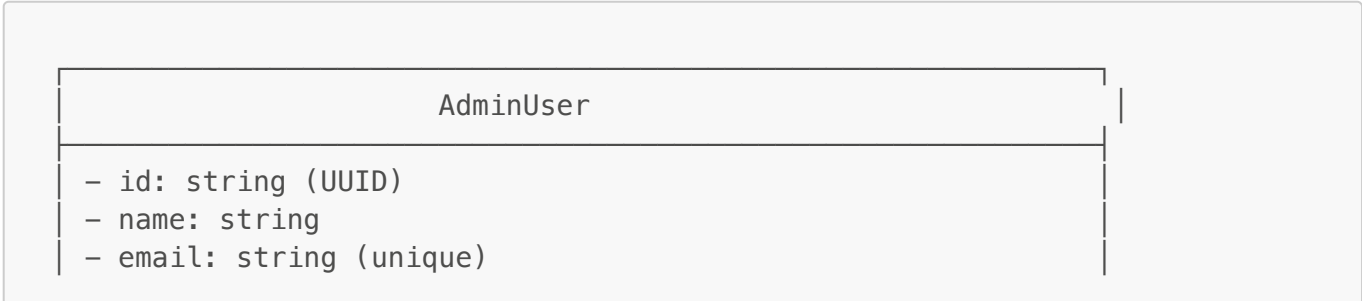
```
┌──────────────────────────────────────────────────────────────────────┐
|                          Quarry                                      |
|----------------------------------------------------------------------|
| - id: string (UUID)                                                  |
| - name: string                                                       |
| - owner_name: string (nullable)                                      |
| - phone: string                                                      |
| - email: string (nullable)                                           |
| - address: text                                                      |
| - state: string                                                      |
| - lga: string                                                        |
| - latitude: decimal(10,8) (nullable)                                 |
| - longitude: decimal(11,8) (nullable)                                |
| - images: json (nullable)                                            |
| - operating_hours: json (nullable)                                   |
| - status: enum(active, inactive)                                     |
| - created_at: datetime                                               |
| - updated_at: datetime                                               |
| - deleted_at: datetime (nullable)                                    |
|----------------------------------------------------------------------|
| + getActiveQuarries(): array                                         |
| + getByState(state): array                                           |
| + getNearby(lat, lon, radius): array                                 |
| + getStatistics(): array                                             |
└──────────────────────────────────────────────────────────────────────┘
                              |
                              | 1
                              |
                              | has many
                              |
                              ▼ *
┌──────────────────────────────────────────────────────────────────────┐
|                       PriceMatrix                                    |
|----------------------------------------------------------------------|
| - id: string (UUID)                                                  |
| - quarry_id: string (FK)                                             |
| - product_id: string (FK)                                            |
```

```
| - loading_point_id: string (FK) (nullable)                    |
| - base_price: decimal(10,2)                                   |
| - price_per_km: decimal(10,2)                                 |
| - min_order_tons: decimal(10,2) (nullable)                    |
| - max_distance_km: decimal(10,2) (nullable)                   |
| - is_active: boolean                                          |
| - effective_from: date (nullable)                             |
| - effective_to: date (nullable)                               |
| - created_at: datetime                                        |
| - updated_at: datetime                                        |
|---------------------------------------------------------------|
| + getPrice(quarryId, productId, loadingPointId): PriceMatrix  |
| + calculateDeliveryPrice(distance, tonnage): decimal          |
| + bulkUpdate(quarryId, data): boolean                         |
└---------------------------------------------------------------┘
```

```
┌---------------------------------------------------------------┐
|                          Order                                |
|---------------------------------------------------------------|
| - id: string (UUID)                                           |
| - order_number: string (unique)                               |
| - user_id: string (FK)                                        |
| - type: enum(material, vehicle_hire)                          |
| - status: enum(pending, confirmed, in_transit, delivered,     |
|               completed, cancelled)                           |
| - delivery_address: text                                      |
| - delivery_state: string                                      |
| - delivery_lga: string                                        |
| - delivery_latitude: decimal(10,8) (nullable)                 |
| - delivery_longitude: decimal(11,8) (nullable)                |
| - estimated_tonnage: decimal(10,2) (nullable)                 |
| - actual_tonnage: decimal(10,2) (nullable)                    |
| - price_before_weight_adjustment: decimal(15,2) (nullable)    |
| - final_amount: decimal(15,2)                                 |
| - payment_method: enum(wallet, cash, bank_transfer)           |
| - payment_status: enum(pending, paid, refunded)               |
| - notes: text (nullable)                                      |
| - scheduled_delivery_date: date (nullable)                    |
| - actual_delivery_date: datetime (nullable)                   |
| - created_at: datetime                                        |
| - updated_at: datetime                                        |
| - deleted_at: datetime (nullable)                             |
|---------------------------------------------------------------|
| + getByUser(userId): array                                    |
| + updateStatus(id, status): boolean                           |
| + updateActualWeight(id, tonnage): boolean                    |
| + getStatistics(startDate, endDate): array                    |
| + getTodayOrders(): array                                     |
└---------------------------------------------------------------┘
                              |
                              | 1
```


has many
OrderItem
- id: string (UUID)
- order_id: string (FK)
- product_id: string (FK)
- quarry_id: string (FK) (nullable)
- loading_point_id: string (FK) (nullable)
- quantity_tons: decimal(10,2)
- unit_price: decimal(10,2)
- distance_km: decimal(10,2) (nullable)
- delivery_charge: decimal(10,2)
- subtotal: decimal(15,2)
- created_at: datetime
- updated_at: datetime
+ calculateSubtotal(): decimal


VehicleType
- id: string (UUID)
- name: string
- capacity_tons: decimal(10,2)
- rate_per_km: decimal(10,2)
- base_charge: decimal(10,2)
- description: text (nullable)
- image: string (nullable)
- is_active: boolean
- created_at: datetime
- updated_at: datetime
- deleted_at: datetime (nullable)
+ getActiveVehicles(): array
+ calculateHirePrice(vehicleId, distance): decimal
+ getStatistics(): array

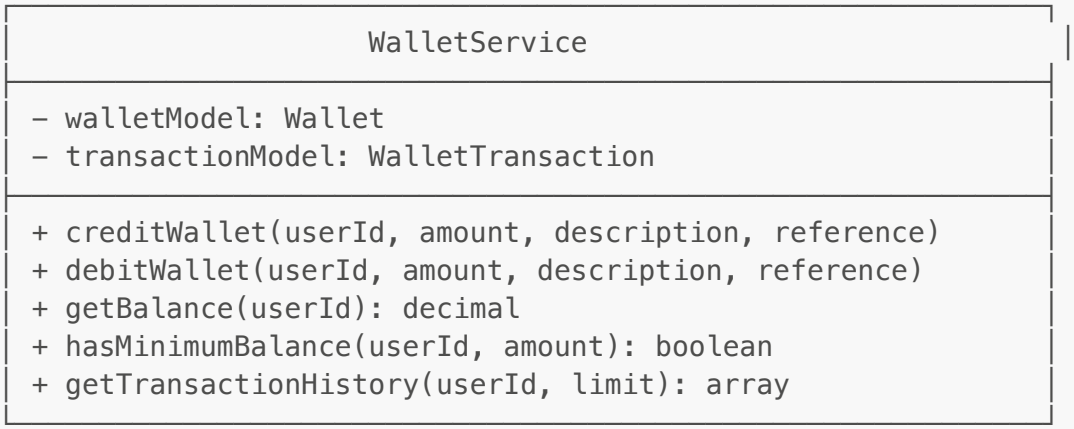

AdminUser
- id: string (UUID)
- name: string
- email: string (unique)

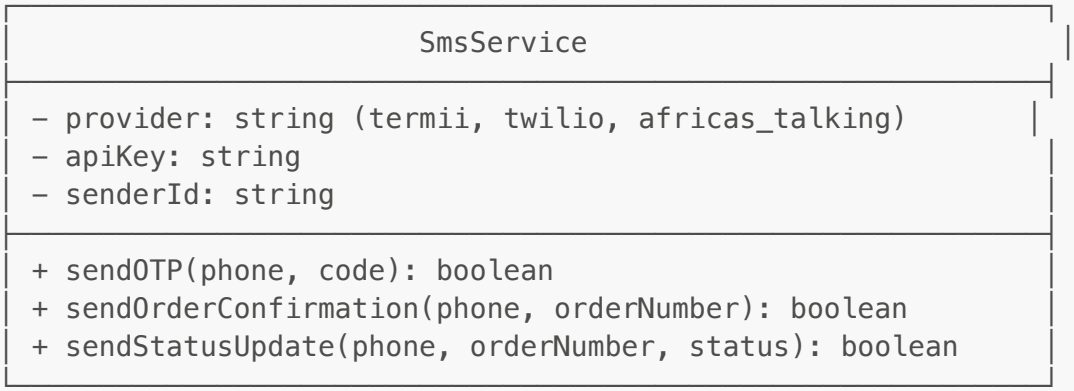
```
| - phone: string (nullable)                                    |
| - password: string (hashed)                                   |
| - role: enum(super_admin, admin, operator, dispatcher)        |
| - permissions: json                                           |
| - status: enum(active, inactive, suspended)                   |
| - last_login_at: datetime (nullable)                          |
| - created_at: datetime                                        |
| - updated_at: datetime                                        |
| - deleted_at: datetime (nullable)                             |
|---------------------------------------------------------------|
| + findByEmail(email): AdminUser                               |
| + findByEmailWithPassword(email): AdminUser                   |
| + updateLastLogin(id): boolean                                |
| + updatePassword(id, hashedPassword): boolean                 |
| + updatePermissions(id, permissions): boolean                 |
| + getStatistics(): array                                      |
+---------------------------------------------------------------+
```

4.2 Service Classes

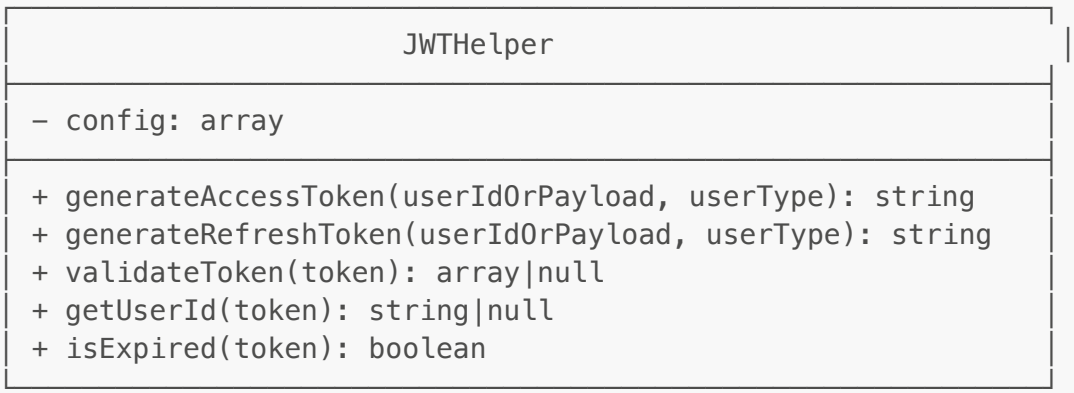
```
+---------------------------------------------------------------+
|                      PricingService                           |
|---------------------------------------------------------------|
| - priceMatrixModel: PriceMatrix                               |
| - productModel: Product                                       |
|---------------------------------------------------------------|
| + calculateOrderPrice(productId, quarryId, loadingPointId,    |
|                       deliveryLat, deliveryLon, tonnage)      |
| + getAvailableQuarries(productId, deliveryLat, deliveryLon)   |
| + calculateDistance(lat1, lon1, lat2, lon2): decimal          |
+---------------------------------------------------------------+
```

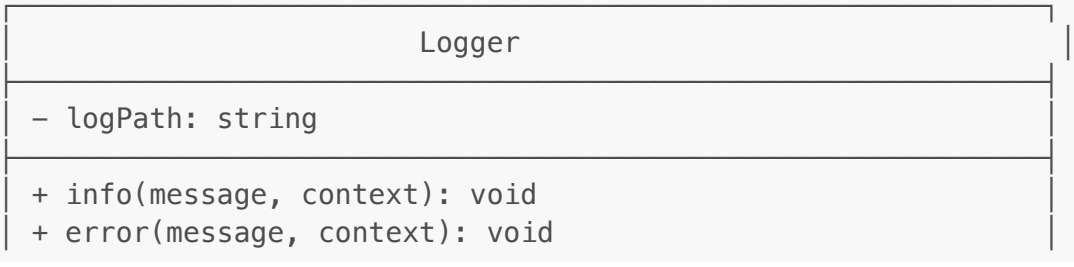
```
+---------------------------------------------------------------+
|                       OrderService                            |
|---------------------------------------------------------------|
| - orderModel: Order                                           |
| - walletService: WalletService                                |
| - pricingService: PricingService                              |
|---------------------------------------------------------------|
| + createMaterialOrder(userId, items, deliveryInfo): Order     |
| + createVehicleHireOrder(userId, vehicleId, route): Order     |
| + updateOrderStatus(orderId, status): boolean                 |
| + handleWeightAdjustment(orderId, actualTonnage): boolean     |
| + cancelOrder(orderId, reason): boolean                       |
+---------------------------------------------------------------+
```





4.3 Helper Classes



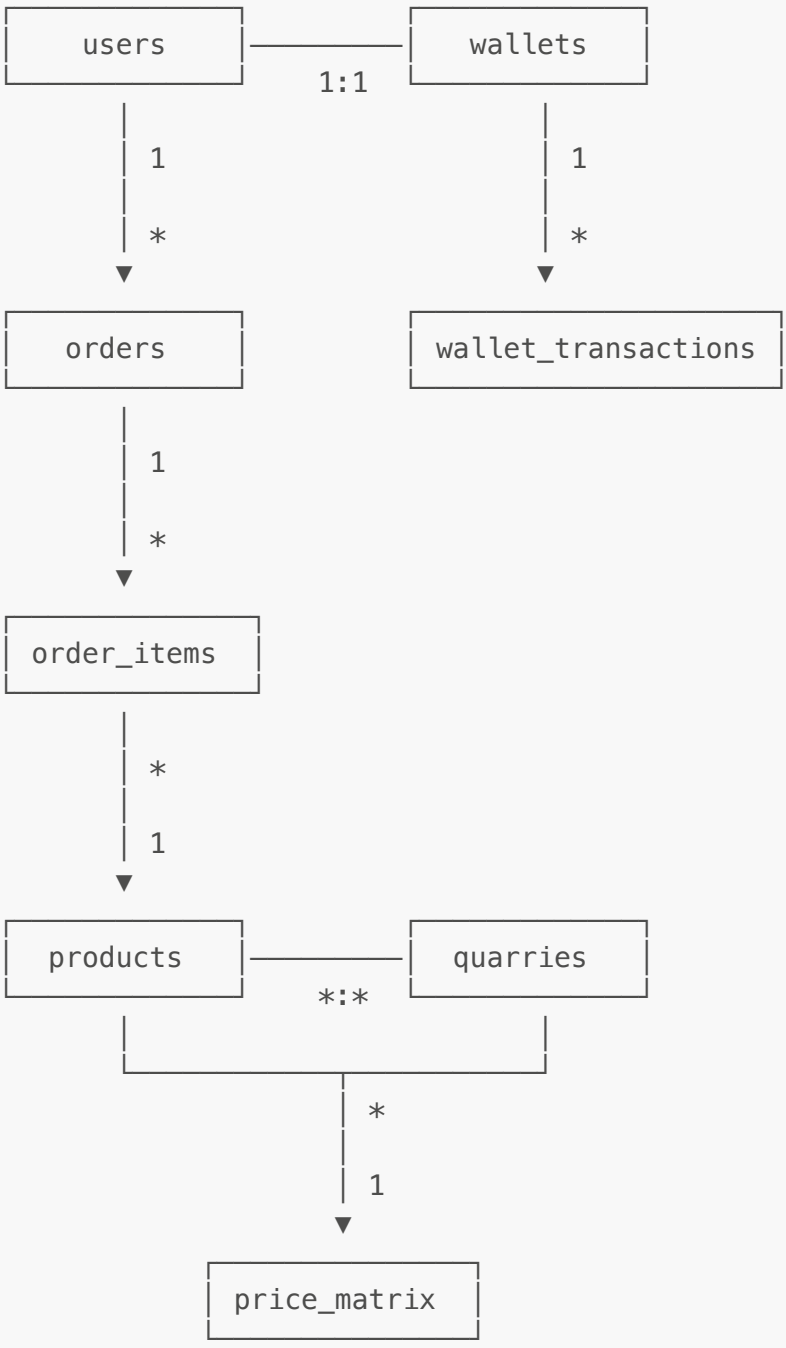


```
| + warning(message, context): void                                    |
| + debug(message, context): void                                      |
| + logRequest(method, uri, statusCode, duration): void                |
└──────────────────────────────────────────────────────────────────────┘
```

\newpage

Database Schema

5.1 Entity Relationship Diagram (ERD)



5.2 Core Tables

users

```
CREATE TABLE users (
    id CHAR(36) PRIMARY KEY,
    phone VARCHAR(20) UNIQUE NOT NULL,
    email VARCHAR(255) UNIQUE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    user_type ENUM('b2b', 'b2c') NOT NULL,
    company_name VARCHAR(255),
    company_address TEXT,
    has_weighbridge BOOLEAN DEFAULT FALSE,
    profile_image VARCHAR(255),
    status ENUM('active', 'suspended', 'inactive') DEFAULT 'active',
    phone_verified_at TIMESTAMP NULL,
    email_verified_at TIMESTAMP NULL,
    last_login_at TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    INDEX idx_phone (phone),
    INDEX idx_email (email),
    INDEX idx_user_type (user_type),
    INDEX idx_status (status)
);
```

orders

```
CREATE TABLE orders (
    id CHAR(36) PRIMARY KEY,
    order_number VARCHAR(20) UNIQUE NOT NULL,
    user_id CHAR(36) NOT NULL,
    type ENUM('material', 'vehicle_hire') NOT NULL,
    status ENUM('pending', 'confirmed', 'in_transit',
                'delivered', 'completed', 'cancelled') DEFAULT 'pending',
    delivery_address TEXT NOT NULL,
    delivery_state VARCHAR(100) NOT NULL,
    delivery_lga VARCHAR(100) NOT NULL,
    delivery_latitude DECIMAL(10, 8),
    delivery_longitude DECIMAL(11, 8),
    estimated_tonnage DECIMAL(10, 2),
    actual_tonnage DECIMAL(10, 2),
    price_before_weight_adjustment DECIMAL(15, 2),
    final_amount DECIMAL(15, 2) NOT NULL,
    payment_method ENUM('wallet', 'cash', 'bank_transfer') DEFAULT
'wallet',
    payment_status ENUM('pending', 'paid', 'refunded') DEFAULT 'pending',
    notes TEXT,
    scheduled_delivery_date DATE,
```

```
    actual_delivery_date TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    FOREIGN KEY (user_id) REFERENCES users(id),
    INDEX idx_user (user_id),
    INDEX idx_status (status),
    INDEX idx_type (type),
    INDEX idx_order_number (order_number),
    INDEX idx_created_at (created_at)
);
```

products

```
CREATE TABLE products (
    id CHAR(36) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    description TEXT,
    category VARCHAR(100) NOT NULL,
    unit ENUM('tons', 'cubic_meters') DEFAULT 'tons',
    base_price_per_ton DECIMAL(10, 2) NOT NULL,
    images JSON,
    specifications JSON,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    INDEX idx_category (category),
    INDEX idx_is_active (is_active)
);
```

quarries

```
CREATE TABLE quarries (
    id CHAR(36) PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    owner_name VARCHAR(255),
    phone VARCHAR(20) NOT NULL,
    email VARCHAR(255),
    address TEXT NOT NULL,
    state VARCHAR(100) NOT NULL,
    lga VARCHAR(100) NOT NULL,
    latitude DECIMAL(10, 8),
    longitude DECIMAL(11, 8),
    images JSON,
    operating_hours JSON,
    status ENUM('active', 'inactive') DEFAULT 'active',
```

```
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,
    INDEX idx_state (state),
    INDEX idx_status (status)
);
```

price_matrix

```
CREATE TABLE price_matrix (
    id CHAR(36) PRIMARY KEY,
    quarry_id CHAR(36) NOT NULL,
    product_id CHAR(36) NOT NULL,
    loading_point_id CHAR(36),
    base_price DECIMAL(10, 2) NOT NULL,
    price_per_km DECIMAL(10, 2) NOT NULL,
    min_order_tons DECIMAL(10, 2),
    max_distance_km DECIMAL(10, 2),
    is_active BOOLEAN DEFAULT TRUE,
    effective_from DATE,
    effective_to DATE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (quarry_id) REFERENCES quarries(id),
    FOREIGN KEY (product_id) REFERENCES products(id),
    FOREIGN KEY (loading_point_id) REFERENCES loading_points(id),
    UNIQUE KEY unique_pricing (quarry_id, product_id, loading_point_id),
    INDEX idx_quarry (quarry_id),
    INDEX idx_product (product_id)
);
```

wallets

```
CREATE TABLE wallets (
    id CHAR(36) PRIMARY KEY,
    user_id CHAR(36) UNIQUE NOT NULL,
    balance DECIMAL(15, 2) DEFAULT 0.00,
    total_credited DECIMAL(15, 2) DEFAULT 0.00,
    total_debited DECIMAL(15, 2) DEFAULT 0.00,
    is_locked BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    INDEX idx_user (user_id)
);
```

\newpage

API Design

6.1 RESTful API Endpoints

Authentication Endpoints

Admin Authentication

```
POST   /api/admin/auth/login           - Admin login
POST   /api/admin/auth/logout          - Admin logout
GET    /api/admin/auth/me              - Get current admin
POST   /api/admin/auth/refresh         - Refresh access token
POST   /api/admin/auth/change-password - Change password
```

Customer Authentication

```
POST   /api/auth/send-otp              - Send OTP to phone
POST   /api/auth/verify-otp            - Verify OTP and login
POST   /api/auth/refresh               - Refresh access token
POST   /api/auth/logout                - Logout
```

Admin Endpoints

Dashboard

```
GET    /api/admin/dashboard                      - Dashboard stats
GET    /api/admin/dashboard/charts/sales         - Sales chart data
GET    /api/admin/dashboard/charts/orders-by-status - Orders by status
GET    /api/admin/dashboard/top-products         - Top selling products
GET    /api/admin/dashboard/top-customers        - Top customers
```

User Management

```
GET    /api/admin/users                          - List all users
GET    /api/admin/users/{id}                     - Get user details
PUT    /api/admin/users/{id}                     - Update user
DELETE /api/admin/users/{id}                     - Delete user
POST   /api/admin/users/{id}/suspend             - Suspend user
POST   /api/admin/users/{id}/activate            - Activate user
POST   /api/admin/users/{id}/wallet/credit       - Credit wallet
POST   /api/admin/users/{id}/wallet/debit        - Debit wallet
```

Order Management

```
GET    /api/admin/orders                          - List all orders
GET    /api/admin/orders/pending                  - Pending orders
GET    /api/admin/orders/statistics               - Order statistics
GET    /api/admin/orders/{id}                     - Get order details
PUT    /api/admin/orders/{id}/status              - Update order status
PUT    /api/admin/orders/{id}/weight              - Update actual weight
PUT    /api/admin/orders/{id}/assign              - Assign order
```

Product Management

```
GET    /api/admin/products                        - List all products
GET    /api/admin/products/statistics             - Product statistics
POST   /api/admin/products                        - Create product
GET    /api/admin/products/{id}                   - Get product details
PUT    /api/admin/products/{id}                   - Update product
DELETE /api/admin/products/{id}                   - Delete product
PUT    /api/admin/products/{id}/pricing           - Update pricing
```

Quarry Management

```
GET    /api/admin/quarries                        - List all quarries
GET    /api/admin/quarries/statistics             - Quarry statistics
POST   /api/admin/quarries                        - Create quarry
GET    /api/admin/quarries/{id}                   - Get quarry details
PUT    /api/admin/quarries/{id}                   - Update quarry
DELETE /api/admin/quarries/{id}                   - Delete quarry
GET    /api/admin/quarries/{id}/pricing           - Get pricing matrix
PUT    /api/admin/quarries/{id}/pricing           - Update pricing
PUT    /api/admin/quarries/{id}/pricing/bulk      - Bulk update pricing
```

Customer Endpoints

Products

```
GET    /api/products                              - Browse products
GET    /api/products/{id}                         - Get product details
GET    /api/products/categories                   - Get categories
GET    /api/products/search                       - Search products
```

Orders

```
GET    /api/orders                              - My orders
POST   /api/orders                              - Create order
GET    /api/orders/{id}                         - Order details
PUT    /api/orders/{id}/cancel                  - Cancel order
GET    /api/orders/{id}/track                   - Track order
```

Wallet

```
GET    /api/wallet                              - Get wallet balance
GET    /api/wallet/transactions                 - Transaction history
POST   /api/wallet/topup                        - Top up wallet
```

Profile

```
GET    /api/profile                             - Get my profile
PUT    /api/profile                             - Update profile
POST   /api/profile/upload-image                - Upload profile image
```

6.2 Request/Response Examples

Admin Login Request

```
POST /api/admin/auth/login

Request:
{
  "email": "admin@charissatics.com",
  "password": "admin123"
}

Response (200 OK):
{
  "success": true,
  "message": "Login successful",
  "data": {
    "admin": {
      "id": "uuid-here",
      "name": "Admin User",
      "email": "admin@charissatics.com",
      "role": "super_admin",
      "permissions": []
    },
```

```
    "tokens": {
      "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
      "token_type": "Bearer",
      "expires_in": 86400
    }
  }
}
```

Create Material Order Request

```
POST /api/orders
Authorization: Bearer {token}

Request:
{
  "type": "material",
  "items": [
    {
      "product_id": "uuid-product-1",
      "quarry_id": "uuid-quarry-1",
      "quantity_tons": 10
    }
  ],
  "delivery_address": "123 Construction Site, Lagos",
  "delivery_state": "Lagos",
  "delivery_lga": "Ikeja",
  "delivery_latitude": 6.5964,
  "delivery_longitude": 3.3454,
  "scheduled_delivery_date": "2026-02-01",
  "payment_method": "wallet",
  "notes": "Please deliver early morning"
}

Response (201 Created):
{
  "success": true,
  "message": "Order created successfully",
  "data": {
    "order": {
      "id": "uuid-order-1",
      "order_number": "ORD-20260127-0001",
      "type": "material",
      "status": "pending",
      "final_amount": 150000.00,
      "payment_status": "paid",
      "created_at": "2026-01-27T10:30:00Z"
    }
  }
}
```

Get Dashboard Stats Request

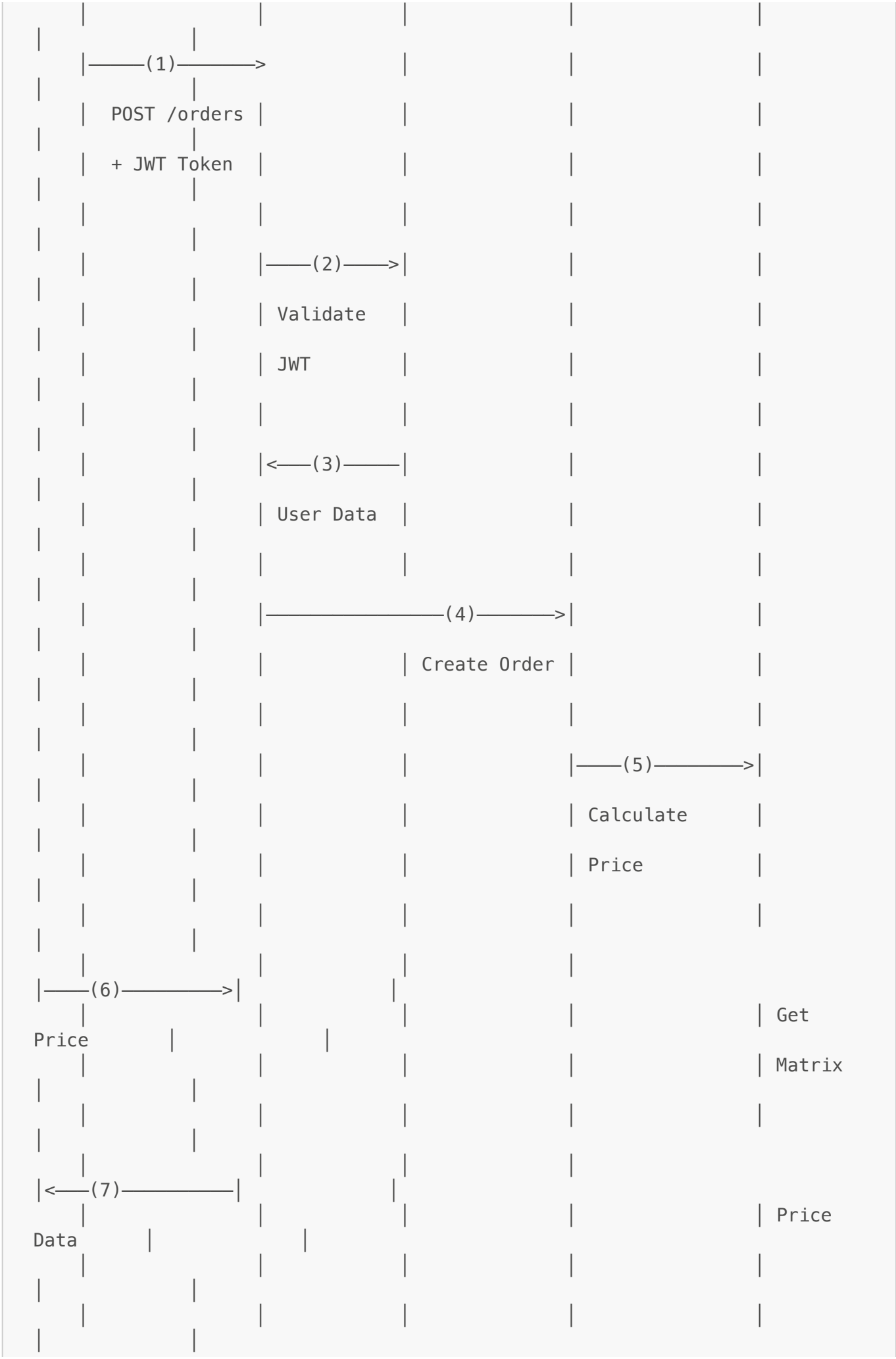
```
GET /api/admin/dashboard
Authorization: Bearer {token}

Response (200 OK):
{
  "success": true,
  "data": {
    "users": {
      "total": 150,
      "b2b": 45,
      "b2c": 105,
      "active": 142,
      "new_this_month": 12
    },
    "orders": {
      "total": 520,
      "completed": 480,
      "cancelled": 15,
      "pending": 25,
      "material_orders": 470,
      "vehicle_hire_orders": 50,
      "today_count": 8
    },
    "revenue": {
      "total": 52500000.00,
      "currency": "NGN"
    },
    "tonnage": {
      "total": 8450.5,
      "unit": "tons"
    },
    "wallets": {
      "total_balance": 15750000.00,
      "average_balance": 105000.00,
      "total_wallets": 150
    }
  }
}
```

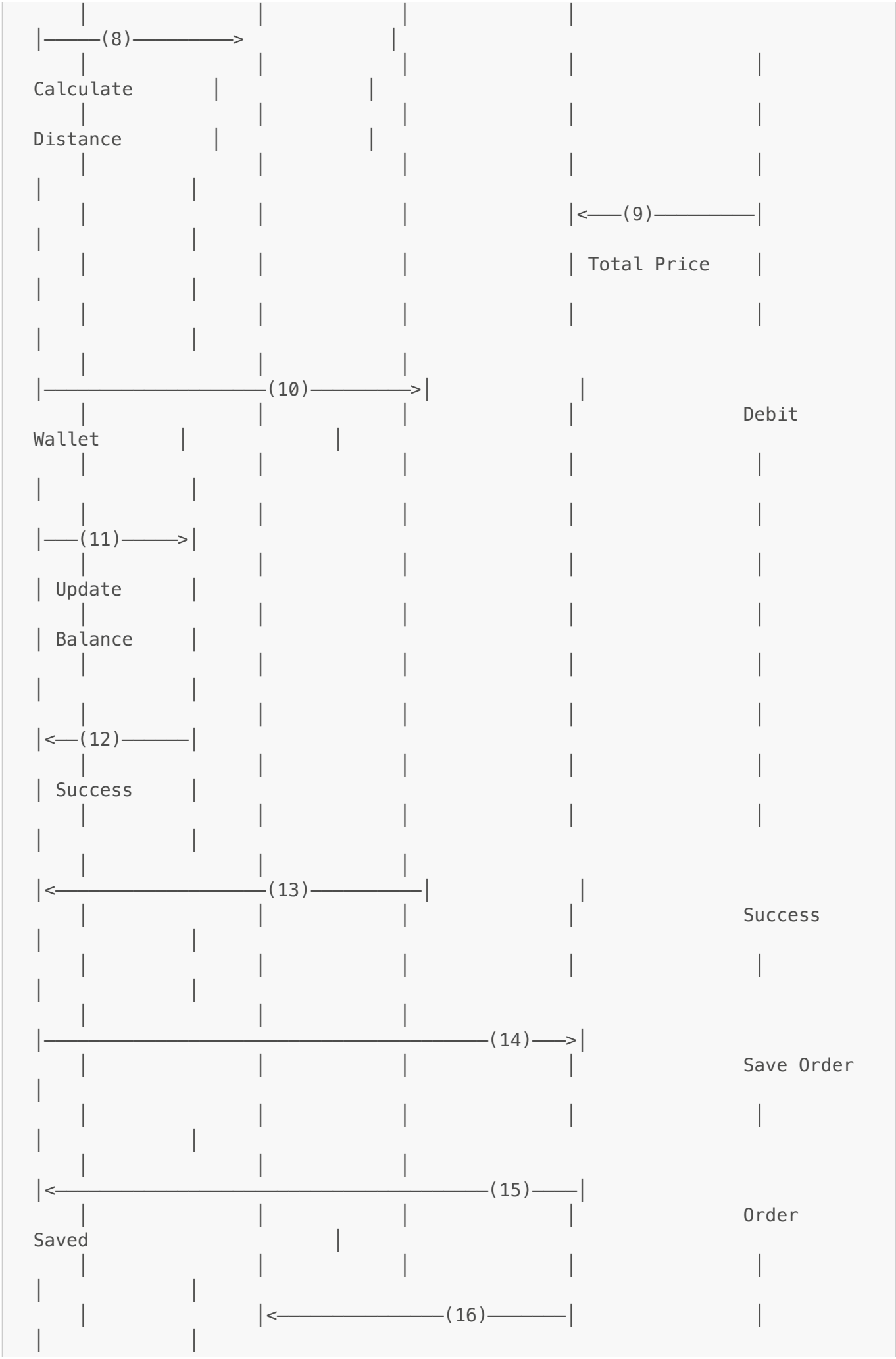
\newpage

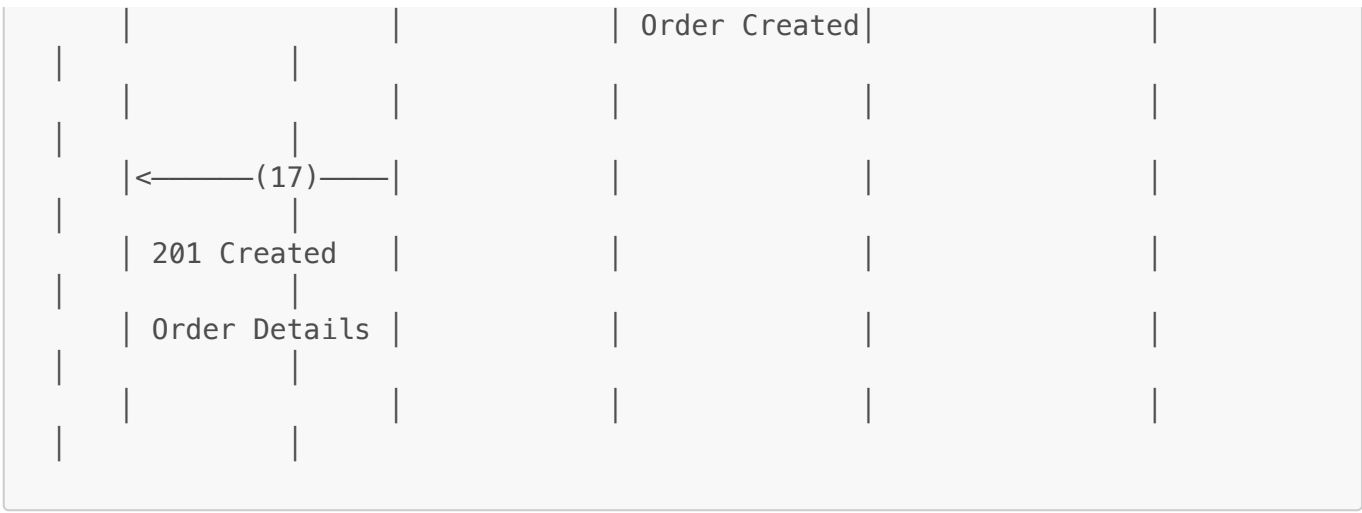
Sequence Diagrams

7.1 Customer Order Placement Flow

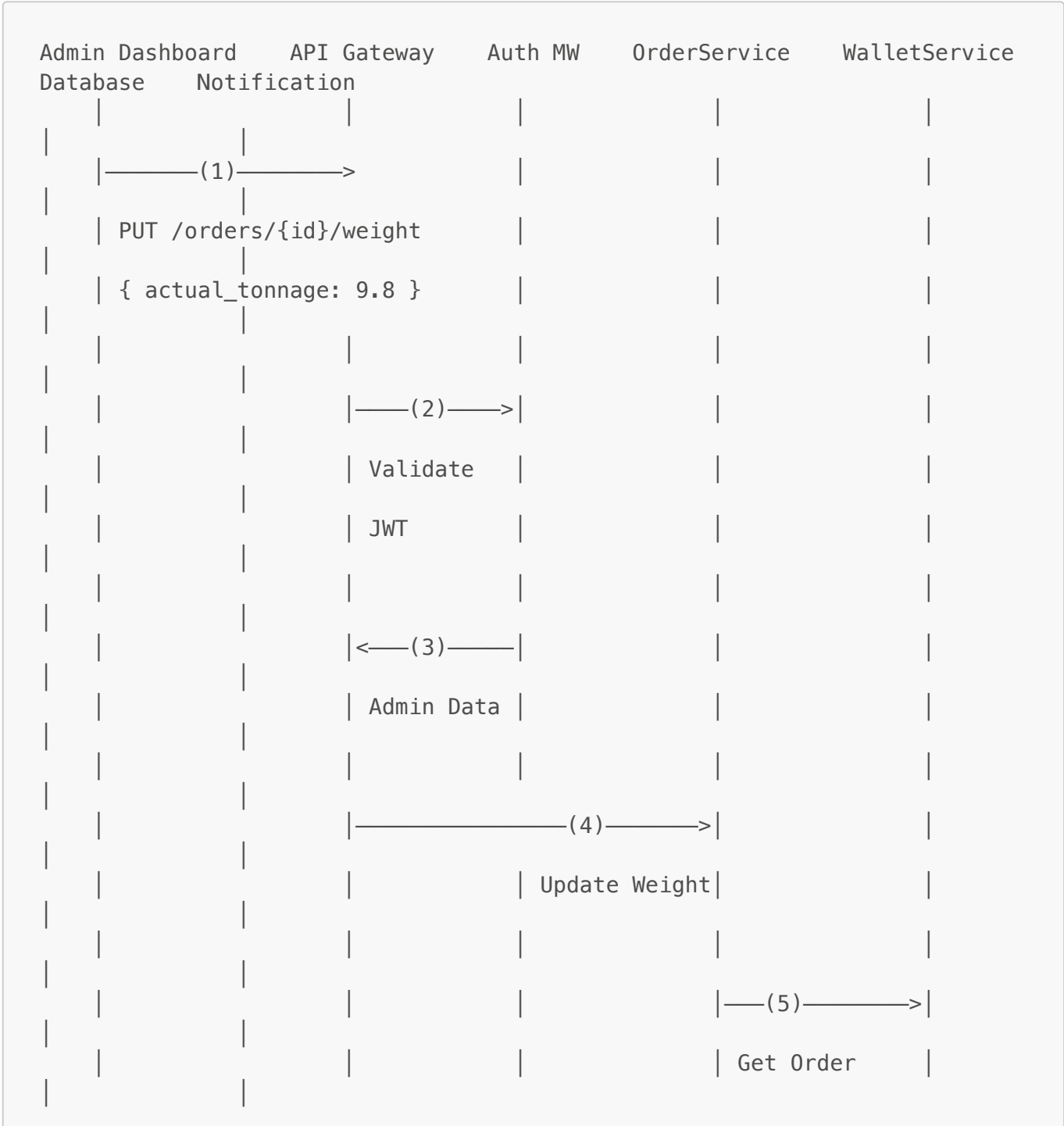
```
Customer App    API Gateway    Auth MW    OrderService    PricingService
WalletService    Database
```

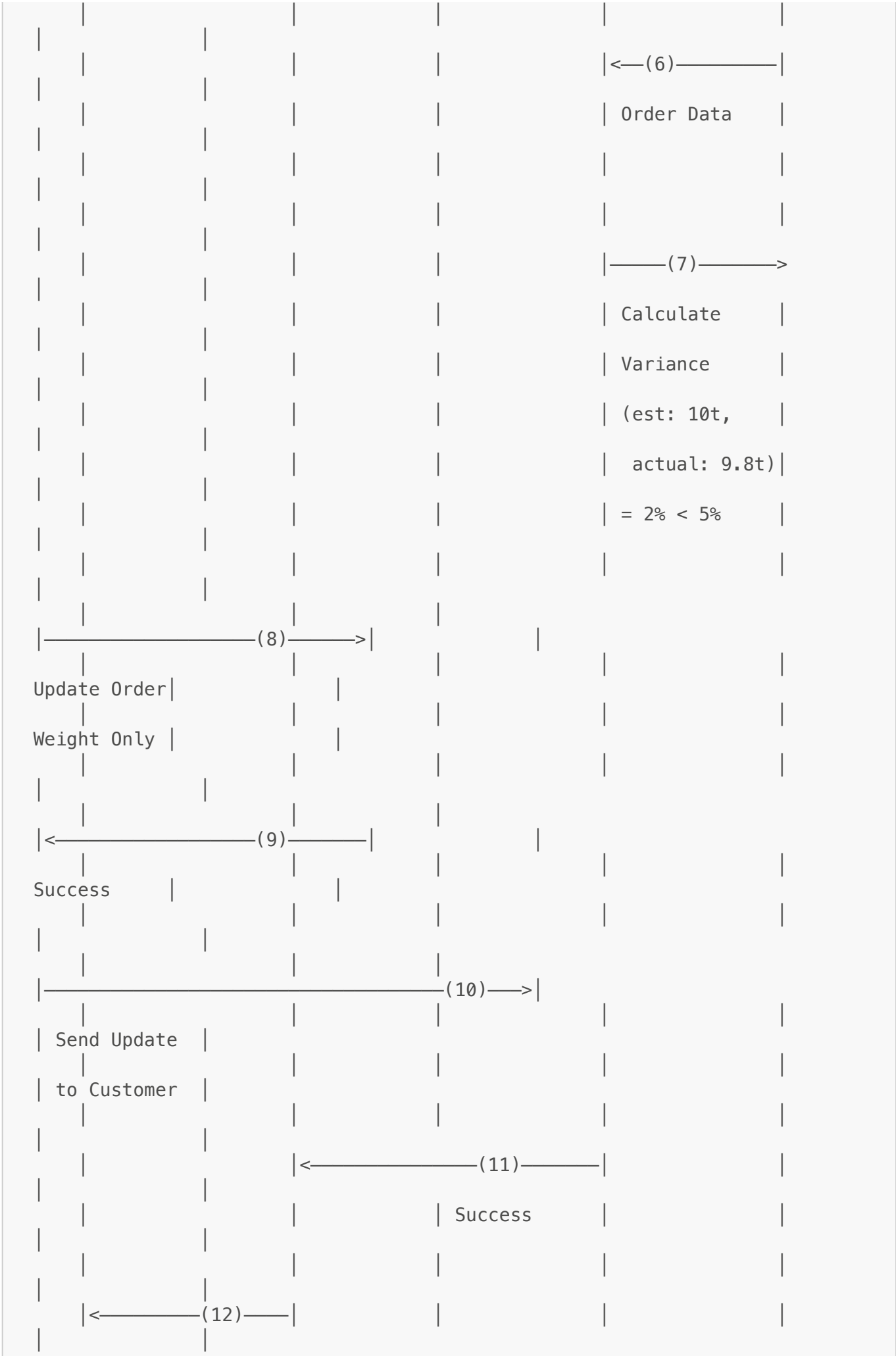

(1)
POST /orders
+ JWT Token
(2)
Validate
JWT
(3)
User Data
(4)
Create Order
(5)
Calculate
Price
(6)
Get
Price
Matrix
(7)
Price
Data


(8)
Calculate Distance
(9)
Total Price
(10)
Debit Wallet
(11)
Update Balance
(12)
Success
(13)
Success
(14)
Save Order
(15)
Order Saved
(16)



7.2 Admin Weight Adjustment Flow




(6)
Order Data
(7)
Calculate
Variance
(est: 10t,
actual: 9.8t)
= 2% < 5%
(8)
Update Order
Weight Only
(9)
Success
(10)
Send Update
to Customer
(11)
Success
(12)

```
    | 200 OK          |           |              |               |
|              |
    | Updated Order   |           |              |               |
|              |
    |                 |           |              |               |
|              |
```

Note: If variance > 5%, system would recalculate price and process refund/additional charge through WalletService.

\newpage

Security Architecture

8.1 Authentication & Authorization

JWT Token Structure

Access Token Payload:

```
{
  "user_id": "uuid-here",
  "sub": "uuid-here",
  "user_type": "admin|customer",
  "iss": "charissatics-api",
  "iat": 1706356800,
  "exp": 1706443200,
  "type": "access"
}
```

Refresh Token Payload:

```
{
  "user_id": "uuid-here",
  "sub": "uuid-here",
  "user_type": "admin|customer",
  "iss": "charissatics-api",
  "iat": 1706356800,
  "exp": 1707048000,
  "type": "refresh"
}
```

Token Lifecycle

1. User authenticates (Admin: email/password, Customer: OTP)
2. System generates access token (1 hour) and refresh token (7 days)

3. Tokens are stored hashed in `user_sessions` table
4. Access token used for API requests
5. When access token expires, use refresh token to get new access token
6. On logout, session is invalidated

Admin Permissions

Roles:

- `super_admin`: Full system access
- `admin`: Most operations except admin user management
- `operator`: Order and inventory management
- `dispatcher`: Order fulfillment and delivery management

Permission Examples:

- `users.view`, `users.edit`, `users.delete`
- `orders.view`, `orders.edit`, `orders.cancel`
- `products.create`, `products.edit`, `products.delete`
- `reports.view`, `reports.export`
- `settings.manage`

8.2 Security Measures

Input Validation

- All request data validated using Validator class
- SQL injection prevention through prepared statements
- XSS prevention through output escaping
- CSRF protection for state-changing operations

Password Security

- Passwords hashed using `password_hash()` (bcrypt)
- Minimum 8 characters requirement
- Admin passwords never returned in API responses

File Upload Security

- File type validation (whitelist)
- File size limits (configurable)
- Random filename generation
- PHP execution disabled in upload directories
- Separate directories for public and private uploads

API Security Headers

```
X-Frame-Options: SAMEORIGIN
X-XSS-Protection: 1; mode=block
```

```
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

Rate Limiting

- OTP requests: 5 per hour per phone number
- Login attempts: 10 per hour per IP
- API requests: 100 per minute per user

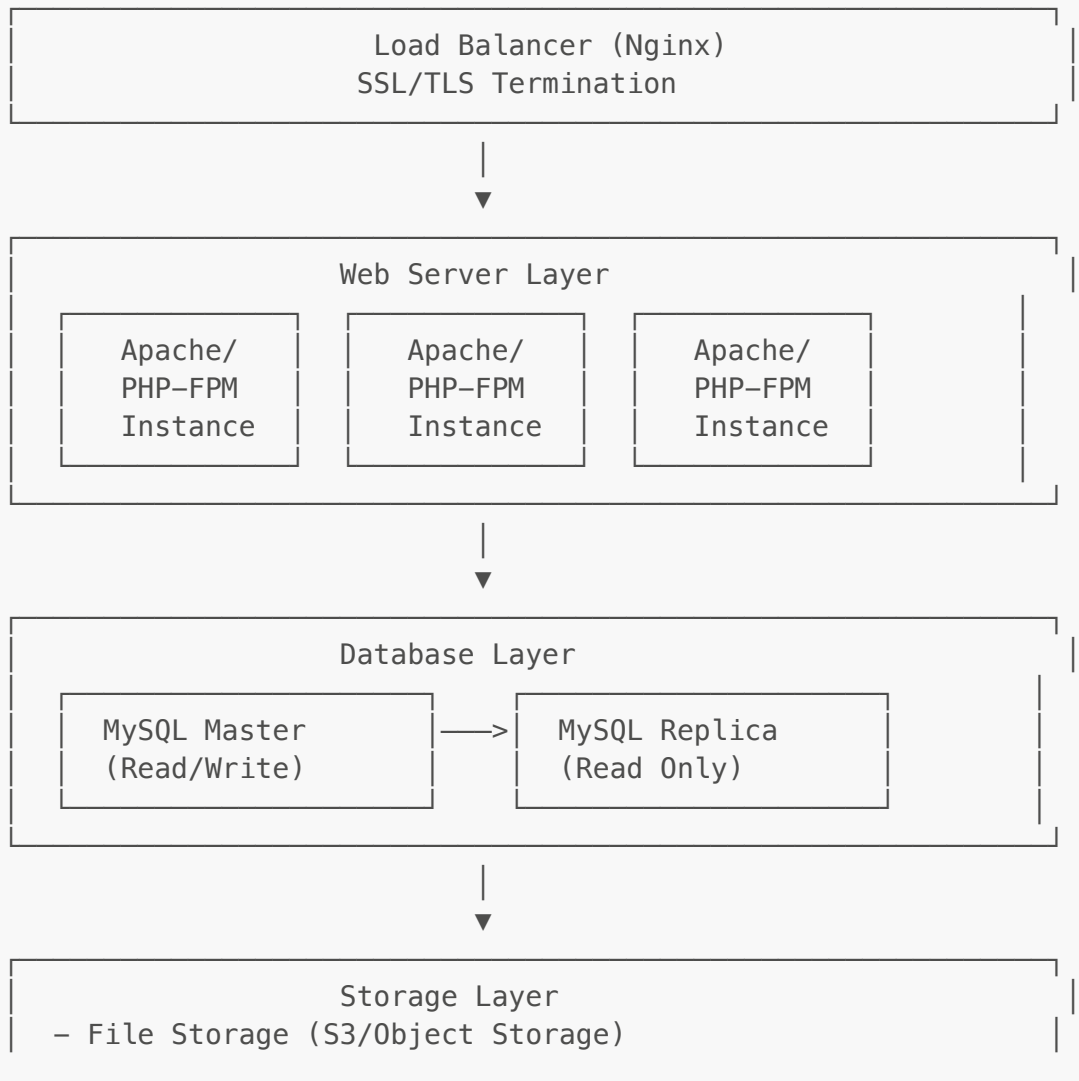
CORS Configuration

- Configurable allowed origins
- Credentials support
- Preflight request handling

\newpage

Deployment Architecture

9.1 Production Environment



```
│  - Logs (ELK Stack/CloudWatch)                                  │
│  - Cache (Redis/Memcached)                                      │
│  - Session Storage (Redis)                                      │
└─────────────────────────────────────────────────────────────────┘
```

9.2 Recommended Server Specifications

Production

- **Web Server**: 2-4 CPU cores, 4-8GB RAM, 50GB SSD
- **Database Server**: 4-8 CPU cores, 8-16GB RAM, 100GB SSD (with daily backups)
- **PHP Version**: 8.0+
- **MySQL Version**: 8.0+
- **Operating System**: Ubuntu 20.04 LTS or higher

Development

- **Local Environment**: PHP 8.0+, MySQL 8.0+
- **Minimum**: 2GB RAM, 10GB free disk space

9.3 Deployment Checklist

- [ ] Set `APP_ENV=production` in `.env`
- [ ] Set `APP_DEBUG=false` in `.env`
- [ ] Generate strong `JWT_SECRET` (min 32 characters)
- [ ] Configure database with proper credentials
- [ ] Set up SSL/TLS certificates
- [ ] Enable HTTPS redirect in `.htaccess`
- [ ] Configure proper file permissions (755 for directories, 644 for files)
- [ ] Set up automated database backups
- [ ] Configure log rotation
- [ ] Set up monitoring and alerting
- [ ] Configure SMS provider (Termii/Twilio)
- [ ] Set up payment gateway (Paystack/Flutterwave)
- [ ] Configure email service
- [ ] Test all critical flows
- [ ] Document admin credentials securely

\newpage

Appendices

Appendix A: Technology Justification

Why Custom PHP Framework?

1. **Simplicity**: No complex framework overhead
2. **Control**: Full control over architecture and features

3. **Performance**: Lightweight and fast
4. **Learning**: Better understanding of MVC concepts
5. **Flexibility**: Easy to modify and extend

Why MySQL?

1. **Reliability**: Proven, stable database system
2. **Performance**: Excellent for transactional workloads
3. **Support**: Wide community support and resources
4. **Features**: Full ACID compliance, foreign keys, transactions
5. **Compatibility**: Works well with PHP

Why JWT?

1. **Stateless**: No server-side session storage needed
2. **Scalable**: Easy to scale horizontally
3. **Mobile-Friendly**: Perfect for mobile app integration
4. **Secure**: Cryptographically signed tokens
5. **Standard**: Industry-standard authentication method

Appendix B: Future Enhancements

Phase 2 Features

- Real-time order tracking with GPS
- Push notifications (Firebase)
- Email notifications
- Payment gateway integration (Paystack, Flutterwave)
- SMS OTP integration (Termii, Twilio)
- Advanced analytics and reporting
- Mobile app for drivers
- Quarry owner portal
- API for third-party integrations

Phase 3 Features

- AI-powered demand forecasting
- Route optimization for deliveries
- Customer loyalty program
- Bulk ordering with special pricing
- Subscription-based delivery service
- Integration with accounting software
- Advanced inventory management
- Multi-language support

Appendix C: Glossary

B2B: Business-to-Business - Companies buying materials **B2C**: Business-to-Consumer - Individual customers **JWT**: JSON Web Token - Authentication token format **OTP**: One-Time Password - Temporary

code sent via SMS **UUID**: Universally Unique Identifier - 36-character unique ID **CRUD**: Create, Read, Update, Delete - Basic database operations **API**: Application Programming Interface **MVC**: Model-View-Controller - Software design pattern **REST**: Representational State Transfer - API architecture style **CORS**: Cross-Origin Resource Sharing - Web security feature

Appendix D: Contact & Support

Project Name: Charissatics Ordering Application **Version**: 1.0 **Last Updated**: January 2026 **Documentation**: See `GETTING_STARTED.md` **API Documentation**: See `backend/routes/admin.php` and `backend/routes/api.php`

End of Technical Design Document